LSH Collision Test Documentation

Locality-Sensitive Hashing Collision Analysis for SMHasher3

SMHasher3 Project

September 29, 2025

Contents

1 Overview **3**
1.1 Purpose . . . 3

2 Mathematical Foundation **3**
2.1 DNA Sequence Representation . . . 3
2.2 Similarity Metrics . . . 3
2.3 Collision Rate Analysis . . . 3

3 Implementation Details **4**
3.1 Test Configuration . . . 4
3.2 Key Size to Sequence Length Mapping . . . 4

4 Algorithm Workflow **4**
4.1 Main Test Function . . . 4
4.2 Implementation Algorithm . . . 5

5 Data Generation and Mutation **6**
5.1 Sequence Generation . . . 6
5.2 Mutation Engine . . . 6

6 Hash Family Testing **6**
6.1 Seed Generation Strategy . . . 6
6.2 Collision Detection . . . 7

7 Output Format and Analysis **7**
7.1 CSV Output Structure . . . 7
7.2 Expected Results . . . 7

8 Performance Considerations **7**
8.1 Computational Complexity . . . 7
8.2 Memory Usage . . . 8

9 Error Handling and Validation **8**
9.1 Bounds Checking . . . 8
9.2 Input Validation . . . 8

10 Integration with SMHasher3 **8**
10.1 Template Instantiation . . . 8
10.2 Command Line Integration . . . 9

11 Conclusion **9**

1 Overview

The `LSHCollisionTest.cpp` file implements a comprehensive testing framework for evaluating the locality-sensitive hashing (LSH) properties of hash functions within the SMHasher3 test suite. This test specifically focuses on analyzing collision rates between similar biological sequences (DNA sequences) to assess how well a hash function preserves locality in the input space.

1.1 Purpose

The primary purpose of this test is to evaluate whether hash functions exhibit appropriate locality-sensitive properties when applied to biological sequence data. In an ideal LSH scenario:

- Similar input sequences should have a higher probability of hash collisions
- Dissimilar sequences should have a lower probability of hash collisions
- The collision probability should correlate with sequence similarity

2 Mathematical Foundation

2.1 DNA Sequence Representation

DNA sequences are represented using a 2-bit encoding scheme:

$$\text{Base Encoding:} \quad A \to 00_2, \quad C \to 01_2, \quad G \to 10_2, \quad T \to 11_2 \tag{1}$$

For a sequence of length n bases, the total storage requirement is:

$$\text{Storage Bytes} = \left\lceil \frac{2n}{8} \right\rceil = \left\lceil \frac{n}{4} \right\rceil \tag{2}$$

2.2 Similarity Metrics

The test uses sequence similarity based on Hamming distance. For two sequences S_1 and S_2 of equal length n:

$$\text{Hamming Distance}(S_1, S_2) = \sum_{i=1}^{n} [S_1[i] \neq S_2[i]] \tag{3}$$

$$\text{Similarity}(S_1, S_2) = 1 - \frac{\text{Hamming Distance}(S_1, S_2)}{n} \tag{4}$$

2.3 Collision Rate Analysis

For a hash family $\mathcal{H}$ and two sequences S_1, S_2, the collision probability is:

$$P_{collision}(S_1, S_2) = \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} [h(S_1) = h(S_2)] \tag{5}$$

3 Implementation Details

3.1 Test Configuration

Key Parameters

- **Key Count**: 1024 sequences per similarity level
- **Hash Count**: 1024 hash functions per sequence pair
- **Key Sizes**: Multiple bit lengths (32, 48, 64, 96, 128, 160*, 256*)
- **Similarity Ranges**: 0.0 to 1.0 in 0.1 increments
- **Output**: CSV file with collision rates

*Available with `--extra` flag for non-slow hash functions

3.2 Key Size to Sequence Length Mapping

The relationship between key bits and DNA sequence length is:

$$\text{Sequence Length} = \frac{\text{Key Bits}}{2} \tag{6}$$

Key Bits	Key Bytes	Sequence Length (bases)
32	4	16
48	6	24
64	8	32
96	12	48
128	16	64
160	20	80
256	32	128

4 Algorithm Workflow

4.1 Main Test Function

The `LSHCollisionTest` function orchestrates the entire testing process:

```
template <typename hashtype>
bool LSHCollisionTest(const HashInfo * hinfo, bool extra, flags_t flags) {
    // Initialize output file
    std::ofstream out_file("collisionResults_" + std::string(hinfo->name) + "_"
    + ".csv");

    // Test multiple key sizes
    result &= LSHCollisionTestImpl<hashtype>(hinfo, 32, seed, flags, out_file);
    result &= LSHCollisionTestImpl<hashtype>(hinfo, 48, seed, flags, out_file);
    result &= LSHCollisionTestImpl<hashtype>(hinfo, 64, seed, flags, out_file);
    result &= LSHCollisionTestImpl<hashtype>(hinfo, 96, seed, flags, out_file);
    result &= LSHCollisionTestImpl<hashtype>(hinfo, 128, seed, flags, out_file);

    if (extra && !hinfo->isVerySlow()) {
        result &= LSHCollisionTestImpl<hashtype>(hinfo, 160, seed, flags,
    out_file);
        result &= LSHCollisionTestImpl<hashtype>(hinfo, 256, seed, flags,
    out_file);
    }
```

```

    return result;
}
```

Listing 1: Main Test Function Structure

4.2 Implementation Algorithm

The core algorithm in `LSHCollisionTestImpl` follows these steps:

1. **Initialization**
   - Calculate sequence length from key bits
   - Set up similarity/dissimilarity vectors
   - Initialize collision rate storage matrix
2. **For each dissimilarity level**
   - Generate random sequences using `DataGeneration`
   - Apply mutations using `MutationEngine`
   - Calculate collision rates across hash family
3. **Data Output**
   - Write collision rates to CSV file
   - Format: rows = similarity levels, columns = trials

```
// For each dissimilarity index
for(dissimilarityIdx = 0; dissimilarityIdx < dissimrates.size(); dissimilarityIdx++) {

    // Generate sequences
    DataGeneration<hashtype> dataGen(&records, sequenceLength, keycount, dataGenSeed);
    dataGen.FillRecords(&records, dataGenSeed);

    // Apply mutations
    MutationEngine<hashtype> mutationEngine(sequenceLength, dissimrates[dissimilarityIdx],
                                            MutationSeed, false, keycount);
    mutationEngine.MutateRecordsWithSubstitutions(&records, distances[dissimilarityIdx]);

    // Compute collision rates
    for(trialIdx = 0; trialIdx < keycount; trialIdx++) {
        float collisionCount = 0;

        // Test across hash family
        for (unsigned i = 0; i < hashcount; i++) {
            hashtype hashOrg, hashMut;
            seed_t currentSeed = currentHashSeed + (HashFamilySeedIncrement * i);

            hash(k1, keybytes, currentSeed, &hashOrg);
            hash(k2, keybytes, currentSeed, &hashMut);

            if (hashOrg == hashMut) {
                collisionCount++;
```

```
            }
        }

        collisionRate = collisionCount / static_cast<float>(hashcount);
        collisionRateVec[dissimilarityIdx][trialIdx] = collisionRate;
    }
}
```

Listing 2: Core Algorithm Structure

5 Data Generation and Mutation

5.1 Sequence Generation

The `DataGeneration` class generates random DNA sequences:

- Uses SMHasher3's `Rand` class for reproducible randomness
- Generates sequences using `SEQ_DIST_1` distribution
- Stores sequences in 2-bit encoded format for efficiency

5.2 Mutation Engine

The `MutationEngine` class introduces controlled mutations:

- Implements substitution-based mutations only
- Uses Fisher-Yates shuffle for random position selection
- Ensures mutations result in different bases
- Maintains exact edit distance control

```
// Fisher-Yates shuffle for position selection
for (int i = SequenceLength - 1; i > 0; i--) {
    int j = r.rand_range(i + 1);
    std::swap(indices[i], indices[j]);
}

// Apply mutations at selected positions
for(int i = 0; i < editDistance; i++) {
    int pos = indices[i];
    uint8_t original_base = extractBase(records, record_idx, pos);
    uint8_t new_base = mutate_base(original_base, rmutatebase);
    setBase(records, record_idx, pos, new_base);
}
```

Listing 3: Mutation Algorithm

6 Hash Family Testing

6.1 Seed Generation Strategy

The test employs a sophisticated seeding strategy to ensure independent hash functions:

$$\text{Hash Family Seed} = \text{Base Seed} + (\text{0x9e3779b9} \times \text{Trial Index}) \tag{7}$$

$$\text{Individual Hash Seed} = \text{Family Seed} + (\text{0x9e3779b9} \times \text{Hash Index}) \tag{8}$$

This ensures that:

- Each trial uses a different family of hash functions
- Each hash within a family is independent
- Results are reproducible with fixed seeds

6.2 Collision Detection

For each sequence pair $(S_{original}, S_{mutated})$ and hash family $\{h_1, h_2, \ldots, h_k\}$:

$$\text{Collision Rate} = \frac{1}{k} \sum_{i=1}^{k} [h_i(S_{original}) = h_i(S_{mutated})] \tag{9}$$

7 Output Format and Analysis

7.1 CSV Output Structure

The test generates a CSV file with the following structure:

Similarity Level	**Trial 1**	**Trial 2**	**...**	**Trial 1024**
0.0 (100% dissimilar)	$\text{rate}_{0,1}$	$\text{rate}_{0,2}$	...	$\text{rate}_{0,1024}$
0.1 (90% dissimilar)	$\text{rate}_{1,1}$	$\text{rate}_{1,2}$	...	$\text{rate}_{1,1024}$
...	...	...	...	...
1.0 (0% dissimilar)	$\text{rate}_{10,1}$	$\text{rate}_{10,2}$	...	$\text{rate}_{10,1024}$

7.2 Expected Results

For an ideal locality-sensitive hash function:

- **High Similarity** (similarity $\to$ 1.0): Collision rate $\to$ 1.0
- **Low Similarity** (similarity $\to$ 0.0): Collision rate $\to \frac{1}{2^{\text{hash bits}}}$
- **Monotonic Relationship**: Collision rate should increase with similarity

8 Performance Considerations

8.1 Computational Complexity

The total computational complexity is:

$$O(S \times K \times H \times L) \tag{10}$$

Where:

- S = Number of similarity levels (11)
- K = Number of key pairs (1024)
- H = Number of hash functions (1024)
- L = Sequence length (varies by key size)

8.2 Memory Usage

Memory requirements include:

- Sequence storage: $2 \times K \times \lceil L/4 \rceil$ bytes
- Collision rate matrix: $S \times K \times 4$ bytes
- Working memory for hash computations

9 Error Handling and Validation

9.1 Bounds Checking

The implementation includes comprehensive bounds checking:

```
// Validate array access bounds
if (trialIdx * keybytes >= records.SeqTwoBitOrg.size() ||
    trialIdx * keybytes >= records.SeqTwoBitMut.size()) {
    printf("ERROR: Index out of bounds! trialIdx=%u, keybytes=%u\n",
           trialIdx, keybytes);
    return false;
}
```

Listing 4: Bounds Validation

9.2 Input Validation

Key parameters are validated during initialization:

- Sequence length > 0 and > 13 (minimum for meaningful analysis)
- Key count > 0
- Error rates in range $[0.0, 1.0]$
- Edit distances $\leq$ sequence length

10 Integration with SMHasher3

10.1 Template Instantiation

The test is instantiated for all supported hash types using the SMHasher3 instantiation system:

```
INSTANTIATE(LSHCollisionTest, HASHTYPELIST);
```

Listing 5: Template Instantiation

This ensures compatibility with:

- 32-bit hash functions
- 64-bit hash functions
- 128-bit hash functions
- Variable-length hash functions

10.2 Command Line Integration

The test is accessible via:

```
./SMHasher3 --test=LSHCollision [hash_name]
./SMHasher3 --test=LSHCollision --extra [hash_name]  # Extended testing
```

11 Conclusion

The LSH Collision Test provides a comprehensive framework for evaluating the locality-sensitive properties of hash functions on biological sequence data. By systematically varying sequence similarity and measuring collision rates across large hash families, it enables quantitative assessment of a hash function's suitability for applications requiring locality preservation.

The test's rigorous statistical approach, comprehensive parameter coverage, and integration with the SMHasher3 framework make it a valuable tool for hash function analysis in bioinformatics and related domains.